

Robot Dynamics Quiz 1

Prof. Marco Hutter, Dr. Jesus Tordesillas
Teaching Assistants: Victor Klemm, Clemens Schwarke,
Jia-Ruei Chiu, Pascal Roth, Kaixian Qu

October 18, 2023

Duration: 1h 15min

Permitted Aids: The exam is open book, which means you can use the script, slides, exercises, etc; The use of internet (besides for licenses) is forbidden; no communication among students during the test is allowed.

1 Instructions

1. Download the ZIP file `RobotDynamics_Quiz1_2023.zip` from Moodle. Extract all contents of this file into a new folder and set MATLAB's¹ current path to this folder.
2. Run `init_workspace` in the Matlab command line.
3. All problem files that you need to complete are located in the `problems` folder.
4. Run `evaluate_problems` to check if your functions run. This script does not test for correctness. You will get 0 points if a function does not run (e.g., for syntax errors).
5. When the time is up, zip the entire folder and name it `ETHStudentID_StudentName.zip`. Submit this zip-file through Moodle under **Midterm Exam 1 Submission**. You should receive a confirmation email.
6. If the previous step did not succeed, you can email your file to `robotdynamics@leggedrobotics.com` from your ETH email address with the subject line `[RobotDynamics] ETHStudentID - StudentName`.
7. **Important:**
 - (a) Implementations outside the provided templates will not be graded and receive 0 points.
 - (b) Helper functions included in the `solutions/pcode` directory are specifically for Questions 3 and 4. Using these functions in the solutions for other questions is prohibited and will receive 0 points.

¹Online version of MATLAB at <https://matlab.mathworks.com/>

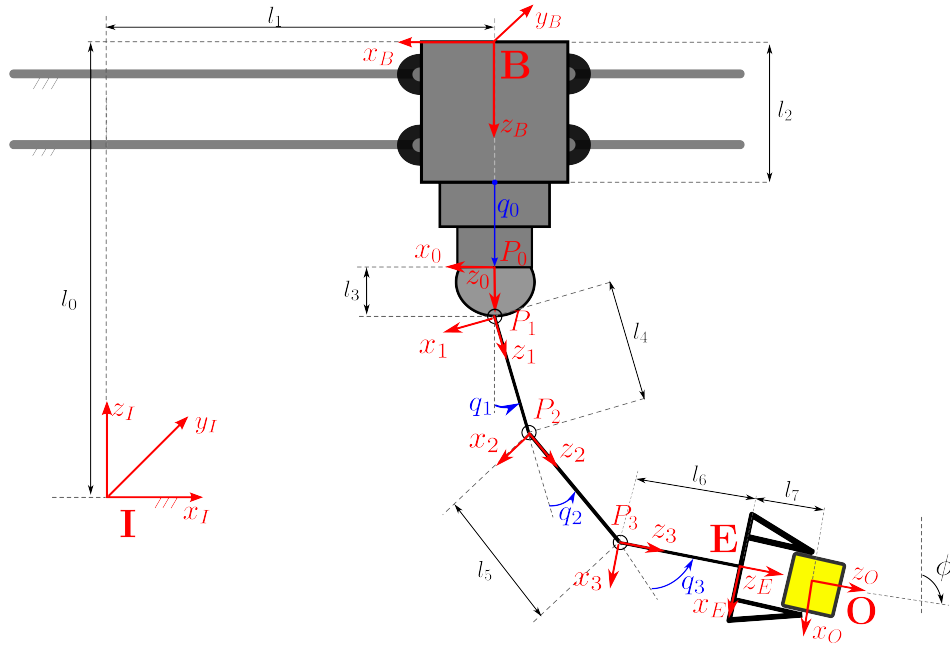


Figure 1: Schematic of an overhead running industrial robot (gantry robot) with a robotic arm of three degrees of freedom attached to a prismatic joint at the base. To simplify the problem, we fix the gantry so that the base does not move. The prismatic joint moves along the z_I direction. All three rotational joints on the arm, indicated with the hollow circles, rotate around the positive y_I axis. The x_B and x_0 are parallel to the x_I axis but oppositely directed.



Figure 2: The Robotic Fabrication Laboratory (RFL) from Gramazio Kohler Research, ETH Zurich.

2 Questions

In this quiz, you will model the forward and differential kinematics for the robotic manipulator shown in Fig. 1, which is adapted and simplified from RFL (Fig. 2). The system has a 3 Degrees-of-Freedom (DoF) arm mounted on a prismatic platform, with 4 DoFs in total. The robot has a gripper attached to the last link of the arm.

The reference frame attached to the base is denoted as $\{B\}$, and it is fixed with respect to the inertial frame $\{I\}$. The first joint corresponds to the prismatic platform that moves linearly along the z_B axis and controls the distance between $\{B\}$ and $\{P_0\}$. The arm is composed of three links. The reference frames attached to each link are denoted as $\{P_1\}$, $\{P_2\}$ and $\{P_3\}$ respectively. The frame attached to the gripper (end-effector) is denoted as $\{E\}$. Additionally, the gripper is grasping an object. The object frame is denoted as $\{O\}$ with an offset l_7 along z_E .

Note: The transformations between $\{P_3\}$ and $\{E\}$ and between $\{E\}$ and $\{O\}$ are fixed.

The generalized coordinates are defined as

$$\mathbf{q} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix} \in \mathbb{R}^4. \quad (1)$$

Clarification 1: The angles q_1, q_2, q_3, ϕ are measured using the right-hand thumb rule. For the state of the scene shown in Fig. 1, q_1, q_2, q_3 would be negative while ϕ is positive (ϕ is defined with respect to the inertial frame I).

In the following questions, all required parameters are passed to your functions in a structure called **params**. You can access it as follows:

```
1 l0 = params.l0;
2 l1 = params.l1;
3 l2 = params.l2;
4 l3 = params.l3;
5 l4 = params.l4;
6 l5 = params.l5;
7 l6 = params.l6;
8 l7 = params.l7;
```

Question 1.

6 P.

Find the homogeneous transformation between the inertial frame $\{I\}$ and the gripper frame $\{E\}$, i.e., the matrix $\mathbf{T}_{IE}$ as a function of the generalized coordinates $\mathbf{q}$.

Hint: You might need to reuse rotation matrices and translation vectors for question 2.

You should implement your solution in the function `jointToGripperPose.m`

Question 2.

5 P.

Compute the position Jacobian ${}_I\mathbf{J}_{IE,P} \in \mathbb{R}^{3 \times 4}$, that fulfills:

$${}_I\mathbf{v}_{IE} = {}_I\mathbf{J}_{IE,P}(\mathbf{q})\dot{\mathbf{q}}, \quad (2)$$

where ${}_I\mathbf{v}_{IE} \in \mathbb{R}^3$ is the linear velocity of point E (the gripper) with respect to a fixed point expressed in frame $\{I\}$.

You should implement your solution in the function `jointToPositionJacobian.m`

Question 3.

4 P.

An important demo of showing the tracking of gripper twist (linear and angular velocities) commands has to happen. However, the motors of the joint q_2 and q_3 are somewhat damaged. The faster they rotate, the more damaged they'll get. Thus, you want to keep them as static as possible.

You decide to apply multi-task differential inverse kinematics control with prioritization. The highest-priority task is to track the desired gripper twist and the second-priority task is to have zero joint velocities for the last two joints.

We denote the 2D gripper velocity with ${}_I\mathbf{w}_{IE,2D} \in \mathbb{R}^3$ and the corresponding desired velocity with ${}_I\mathbf{w}_{IE,2D}^* \in \mathbb{R}^3$, both representing velocity in $x_I - z_I$ plane.

$${}_I\mathbf{w}_{IE,2D} = \begin{bmatrix} {}_I\dot{x}_{IE} \\ {}_I\dot{z}_{IE} \\ \dot{\phi} \end{bmatrix}, \quad {}_I\mathbf{w}_{IE,2D}^* = \begin{bmatrix} {}_I\dot{x}_{IE}^* \\ {}_I\dot{z}_{IE}^* \\ \dot{\phi}^* \end{bmatrix}. \quad (3)$$

where the angle ϕ is indicated in Fig. 1.

For this question, we provide:

- the 2D Jacobian ${}_I\mathbf{J}_{IE,2D} \in \mathbb{R}^{3 \times 4}$, that fulfills:

$${}_I\mathbf{w}_{IE,2D} = {}_I\mathbf{J}_{IE,2D}\dot{\mathbf{q}}. \quad (4)$$

You can call it with `jointTo2DGripperJacobian_solution(q, params);`

- a function to compute the damped pseudo-inverse of a matrix $\mathbf{A}$. You can call it with `pseudoInverseMat_solution(A, lambda).`

You should implement your solution in the function `gripperToJointPrioritized.m`.

Question 4.

5 P.

Just like every other important demo, this robot breaks before the demo and somehow magically works again right afterward. Now that all motors are functioning well, you decide to do something cooler.

Implement a kinematic trajectory controller for the robot arm to manipulate the grasped object by following a given object reference trajectory. Your implementation should return the desired joint velocities $\dot{\mathbf{q}}^*$ given the desired object pose and twist.

Note: This question is not a follow-up of question 3.

Hint: We assume the grasp is rigid, not slipping, and the object is massless. Based on these assumptions, a common practice of tracking the object motion is to transform the object target (pose and twist) to the gripper target and only track the gripper target. Please follow these two steps to implement the trajectory control.

We indicate the object 2D pose and twist with ${}_I\mathbf{p}_{IO,2D} \in \mathbb{R}^3$ and ${}_I\mathbf{w}_{IO,2D} \in \mathbb{R}^3$. Both vectors only consider the $x_I - z_I$ plane.

$${}_I\mathbf{p}_{IO,2D} = \begin{bmatrix} {}_Ix_{IO} \\ {}_Iz_{IO} \\ \phi \end{bmatrix}, \quad {}_I\mathbf{w}_{IO,2D} = \begin{bmatrix} {}_I\dot{x}_{IO} \\ {}_I\dot{z}_{IO} \\ \dot{\phi} \end{bmatrix}, \quad (5)$$

where the angle ϕ is indicated in Fig. 1.

The desired object pose ${}_I\mathbf{p}_{IO,2D}^*$ and twist ${}_I\mathbf{w}_{IO,2D}^*$, as well as the current joint angles $\mathbf{q}$, are passed as inputs to your matlab function. Furthermore, the desired object trajectory does not involve angular movement on the object, meaning that the desired yaw angle is always the initial angle, and the desired angular velocity is 0.

For this question, we provide:

- a function to calculate the current 2D gripper position in the $x_I - z_I$ plane:

$${}_I\mathbf{r}_{IE,2D} = \begin{bmatrix} {}_Ix_{IE} \\ {}_Iz_{IE} \end{bmatrix} \in \mathbb{R}^2. \quad (6)$$

You can call it with `jointTo2DGripperPosition_solution(q, params);`

- the 2D Jacobian ${}_I\mathbf{J}_{IE,2D} \in \mathbb{R}^{3 \times 4}$, that fulfills:

$${}_I\mathbf{w}_{IE,2D} = {}_I\mathbf{J}_{IE,2D}\dot{\mathbf{q}}. \quad (7)$$

You can call it with `jointTo2DGripperJacobian_solution(q, params);`

- A function to compute the damped pseudo-inverse of a matrix $\mathbf{A}$. You can call it with `pseudoInverseMat_solution(A, lambda)`.

You should implement your solution in the function `kinematicTrajectoryControl.m`.

Your implementation is judged based on how well the robot tracks a predefined trajectory.

A kinematics-simulator is implemented for you in `main_control_loop.m`, where the variable `use_solution` is set. You can run this file to visualize the result. If `use_solution` is set to 1, executing this function will show you how the solution should look like. If `use_solution` is set to 0, at each time-step the simulator will use the velocities returned by your own implementation. `animate_robot_arm` is called internally inside `track_reference_trajectory`, and you don't need to call it explicitly.